Activity 5 Report: Database Maintenance and Monitoring

COMP 018 Database Administration | Project tested: Park It (Parking Management System) (PHP + PostgreSQL) | Date of testing: 7 November 2026

1. Cover and links

Item	Value
Testing group	G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph
Project tested	Park It (Parking Management System)
Owning group	Group 2 (member names are not listed in the project files)
Repository	park-it-database (project zip park-it-database-main; the repository link is not included in the files we received)
Tracker	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests
Evidence files	a5_health.txt, a5_logs.txt, a5_maintenance.txt

Every result below comes from this run on a freshly created database.

Run at about 00:41 on 7 November 2026 (Asia/Manila), after the application and access tests.

2. Health and activity

Check	Result
Version	PostgreSQL 18.6, x86_64 Windows
Database size	8446 kB
Connections	1 backend (the monitoring session). The PHP built-in server opens a connection per request. max_connections is 100
Transactions	3,769 commits, 29 rollbacks
Cache	249,593 buffer hits against 9 reads, about 99.996% hit ratio
Deadlocks and temp files	0 and 0
Autovacuum	on, and it had already run: autovacuum on transaction at 00:41:00, autoanalyze on transaction and vehicle at 00:36
Rows	vehicle 66, transaction 19, payment 12, staff_user 2, parking_zone 1
Table size	40 to 56 kB per table, mostly index pages

Dead tuples before maintenance: parking_zone 33 and vehicle 7. The parking_zone row is updated on every entry, payment and recount, so it collects dead versions the fastest. Index use: vehicle_pkey 450 scans and parking_zone_pkey 277 scans, staff_user_username_key 23 (logins). parking_zone_zone_name_key and staff_user_pkey were never used, which is normal for unique and key-only indexes.

3. Logging

Setting	Value	Comment
log_statement	none	No statement logging
log_min_duration_statement	-1	Slow queries are not logged
Server log folder	C:/Program Files/PostgreSQL/18/data/log	Readable
Application log	Console only	The PHP server wrote 32 request lines in its console log during the restore test and no warnings or notices. The app has no log file of its own

4. Log findings

The server log is shared by everything on this server, so earlier entries from other projects also appear. Only entries from 00:35 on 7 November belong to Park It.

Time	Entry	Explanation
00:35:14	new row for relation "parking_zone" violates check constraint "parking_zone_check"	Our direct SQL test that set the count to 51
00:36:35	role "parkit_runtime_grp" and "parkit_app" already exist	The second run of the role SQL
00:36:35 to 00:36:40	permission denied for table staff_user, payment, transaction, vehicle; for sequence vehicle_vehicle_id_seq; for schema public; must be owner; permission denied to create role and for pg_authid; vacuum skipped; FATAL permission denied for database "postgres"	The access tests of Activity 4, all expected
00:40:48	permission denied for sequence vehicle_vehicle_id_seq	The app trying to register a vehicle as parkit_app before the sequence grant (Activity 4)
00:40:48	duplicate key value violates unique constraint "vehicle_license_plate_key"	Our test SQL re-inserting a plate that already existed
00:40:49	permission denied for table vehicle	The app trying to delete a vehicle as parkit_app
00:41:32	permission denied to vacuum and to analyze "vehicle", skipping it	parkit_app attempting maintenance (section 5)

Every entry has a cause in the test steps. Note that no error from the application's own validation (bad plates, wrong payment methods) appears in the database log, because the PHP code rejects those before they reach PostgreSQL.

5. Maintenance performed

Action	Result
ANALYZE (whole database)	Completed, last_analyze set on all 5 tables
VACUUM (VERBOSE, ANALYZE) parking_zone	33 dead row versions removed, 1 remains
VACUUM (VERBOSE, ANALYZE) vehicle	7 dead row versions removed, 66 remain, 2 indexes cleaned
Dead tuples afterwards	parking_zone 0, vehicle 0, all tables 0
VACUUM and ANALYZE as parkit_app	Skipped: permission denied (not the table owner). The project defines no maintenance role

6. Query plans

The four queries below are the ones the pages run: the active-plate search (Module 2), the occupancy recount (runs on every dashboard and monitor refresh), the parked list, and the payment history. They were run with EXPLAIN (ANALYZE, BUFFERS).

Query	Plan	Planning / execution
Active transaction for a plate	Sequential scans with a join on vehicle_id	2.90 ms / 0.05 ms
Occupancy recount (zone left join active transactions)	Sequential scans, hash aggregate	2.52 ms / 0.06 ms
Currently parked list	Sequential scans and a hash join, sort by entry time	2.49 ms / 0.09 ms
Payment history	Three sequential scans joined with hash joins, sort	4.07 ms / 0.11 ms

With 66 vehicles and 19 transactions a sequential scan is the right plan. With sequential scans switched off for one rolled-back transaction, the plate search used vehicle_license_plate_key, so the unique index does serve the lookup when the data is large enough.

Foreign key columns without an index: transaction.vehicle_id and transaction.zone_id. The recount query filters transaction by status = Active, and the payment history and plate search join on vehicle_id, so these are the two tables that grow. The recount runs on every page load and on the monitor's automatic refresh.

7. Recommendations

- Create a maintenance role (member of pg_maintain) for scheduled VACUUM and ANALYZE, because parkit_app cannot run them.
- Set log_min_duration_statement to about 500 ms and log_statement to ddl.
- Add indexes on transaction(vehicle_id) and a partial index on transaction(zone_id) WHERE status = 'Active', before the history grows.
- Reduce the write load of the recount: update parking_zone only when the counter really changes (it already does) and consider reading the count from the transactions instead of keeping a second copy.
- Give the PHP side a log file (error_log) and log the real database error instead of showing it on the page.
- Re-run EXPLAIN when there are tens of thousands of transactions.

8. Feedback for the owners

No health problems found. The useful items are the two unindexed foreign keys, the maintenance role, and a log file for the application.

AI usage declaration

- Tool: Claude (Anthropic).
- Tasks: interpreting the results of the testing.

- Validation: the commands were run by the group on its own computer, and every result in this report comes from the saved output files.